

Password Entry System with 4×4 Keypad and 7-Segment Display

Target board: NUCLEO-L476RG

Development approach: blank STM32CubeIDE project + custom register-level drivers

What this document does: It explains how to create the project, wire the hardware, understand the register mapping, and read every function in every source file. The goal is to make the code easy to teach in a lab or classroom.

System overview: keypad-controlled code checker

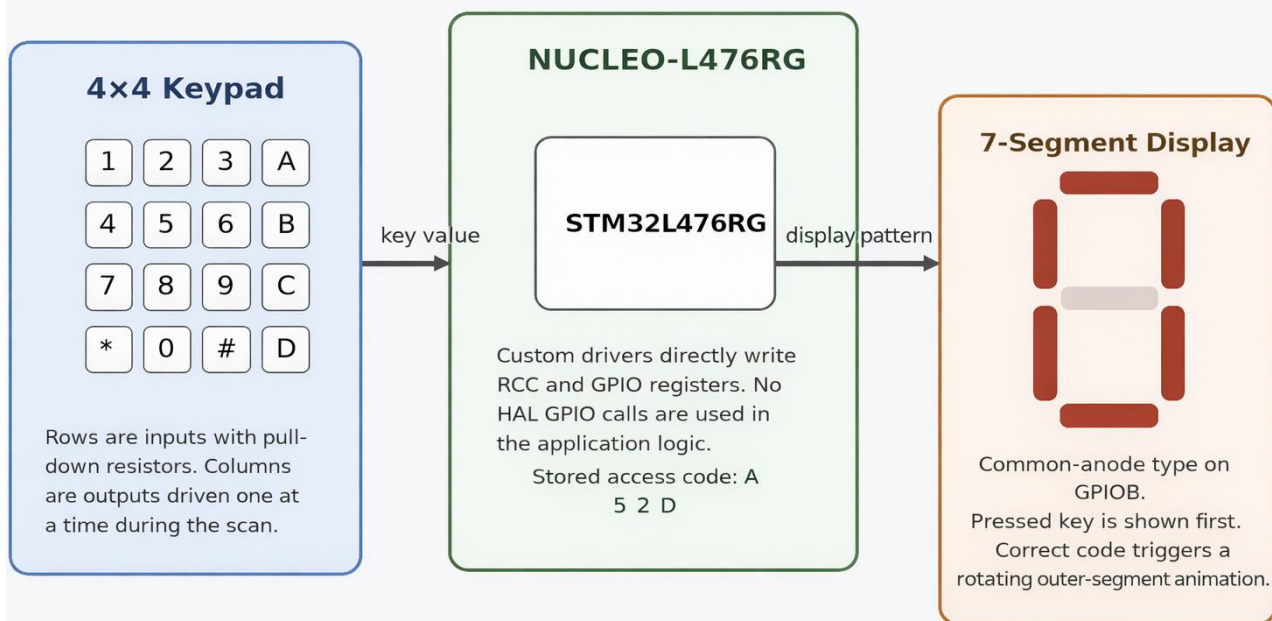


Figure 1. High-level view of the keypad, the NUCLEO-L476RG, and the 7-segment output.

1. Project overview

This project implements a simple access-code system on the STM32L476RG microcontroller. A 4×4 keypad is scanned through GPIOA, a single common-anode 7-segment display is driven through GPIOB, and the main program checks whether the user entered the correct four-character code A-5-2-D. If the code is correct, the display shows a rotating segment animation; otherwise, it briefly shows an error symbol.

By the end of the exercise, students should be able to:

- Create a minimal STM32CubeIDE project for the NUCLEO-L476RG.
- Add custom .h and .c files to a CubeIDE project.
- Explain how memory-mapped registers are represented with C structures.
- Configure GPIO pins as inputs or outputs without using HAL GPIO functions.
- Implement keypad scanning by driving one column at a time and reading row inputs.
- Drive a common-anode 7-segment display with raw bit patterns.
- Follow the full program flow from key press to password validation.

Wiring map used by the project

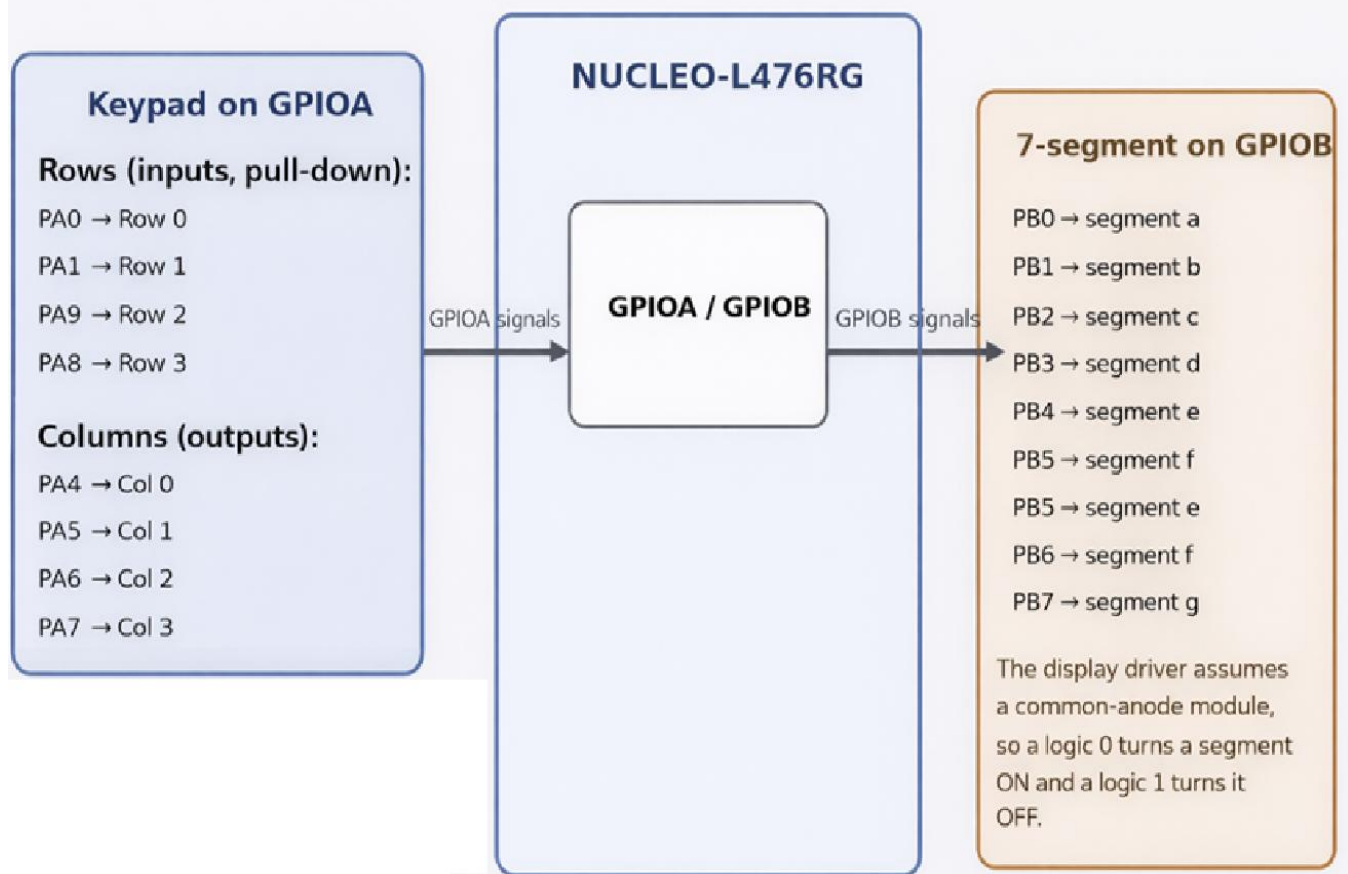


Figure 2. Pin usage in the uploaded project.

2. Hardware connections

A minimal lab setup is enough for this tutorial:

- 1 × NUCLEO-L476RG development board
- 1 × 4×4 matrix keypad
- 1 × common-anode 7-segment display
- Current-limiting resistors for the 7-segment display segments
- Breadboard and jumper wires
- USB cable for programming and power

2.1 Keypad wiring

Signal	MCU pin	Role in software
Row 0	PA0	Input with pull-down
Row 1	PA1	Input with pull-down
Row 2	PA9	Input with pull-down
Row 3	PA8	Input with pull-down
Column 0	PA4	Output driven high during scan
Column 1	PA5	Output driven high during scan
Column 2	PA6	Output driven high during scan
Column 3	PA7	Output driven high during scan

2.2 7-segment display wiring

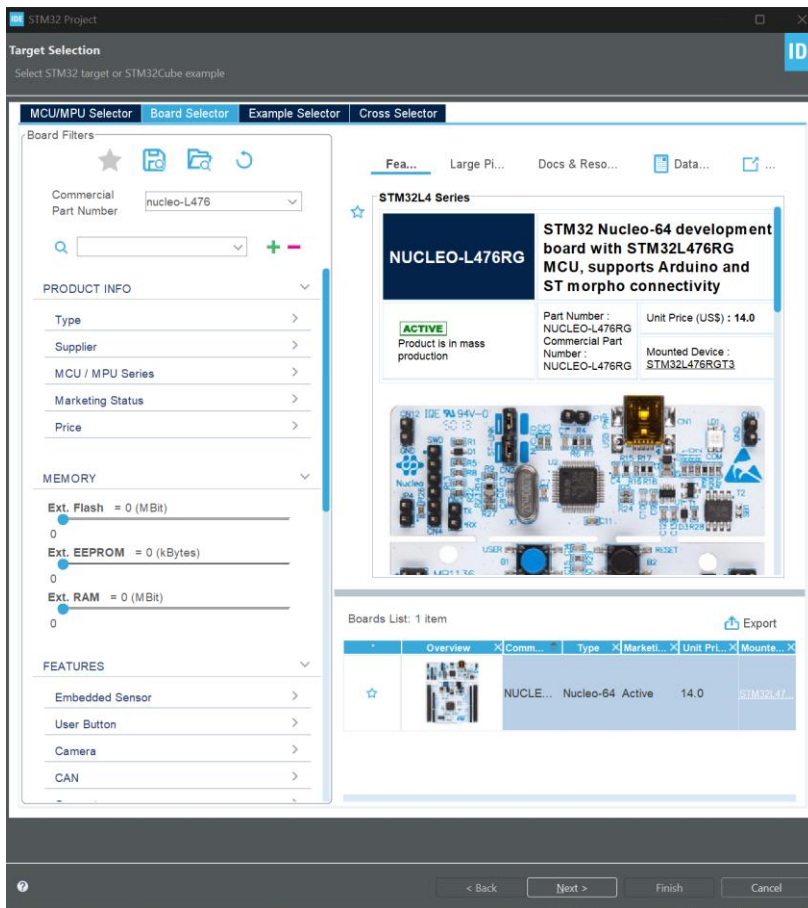
Segment	MCU pin	Meaning of bit position
a	PB0	bit 0
b	PB1	bit 1
c	PB2	bit 2
d	PB3	bit 3
e	PB4	bit 4
f	PB5	bit 5
g	PB6	bit 6

Display polarity: The driver was written for a common-anode display. That is why `SevenSeg_DisplayRaw()` treats 0 as segment ON and 1 as segment OFF.

3. How students can create a blank STM32CubeIDE project

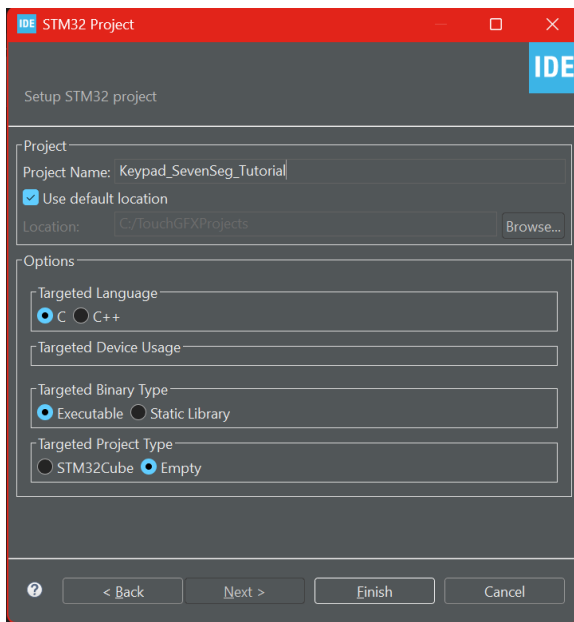
Step 1 - Create the project

Open STM32CubeIDE, select File > New > STM32 Project, and use the Board Selector to choose NUCLEO-L476RG > Next.



Step 2 - Name the project

Give the project a clear name such as Keypad_SevenSeg_Tutorial. Make the toolchain as Empty instead of STM32CubeIDE.

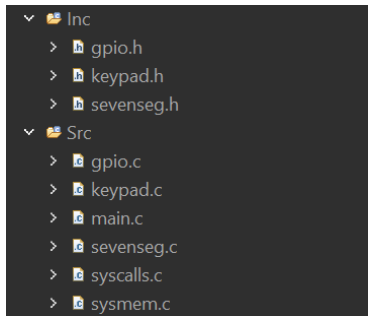


Step 3 - Add the header files

Copy gpio.h, keypad.h, and sevenseg.h into Core/Inc.

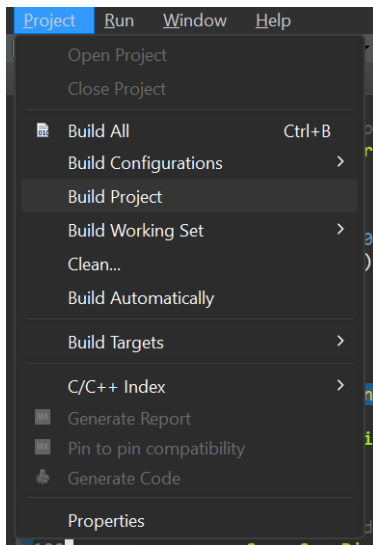
Step 4 - Add the source files

Copy gpio.c, keypad.c, sevenseg.c, and main.c into Core/Src. Replace the generated main.c with the project version.



Step 5 - Build the project

Right-click the project and choose Build project.



Step 6 - Program and test

Flash the board, connect the keypad and display, and test correct and incorrect passwords.

3.1 Where each file should be placed

gpio.h	Core/Inc	Public register and GPIO declarations
keypad.h	Core/Inc	Public keypad interface
sevenseg.h	Core/Inc	Public 7-segment interface
gpio.c	Core/Src	GPIO driver implementation
keypad.c	Core/Src	Keypad scanning implementation
sevenseg.c	Core/Src	7-segment implementation
main.c	Core/Src	Application logic

Why use a blank project first: CubeIDE still provides useful startup files, the linker script, CMSIS device

support, and the debugger configuration. Students then learn how the application works at register level instead of relying on autogenerated GPIO setup.

4. Register mapping and memory-mapped I/O

The most important teaching idea in this project is that peripherals are accessed through fixed memory addresses. The code in `gpio.h` creates C structures whose members line up with the real hardware registers inside RCC and GPIO peripherals.

Register mapping used by the drivers

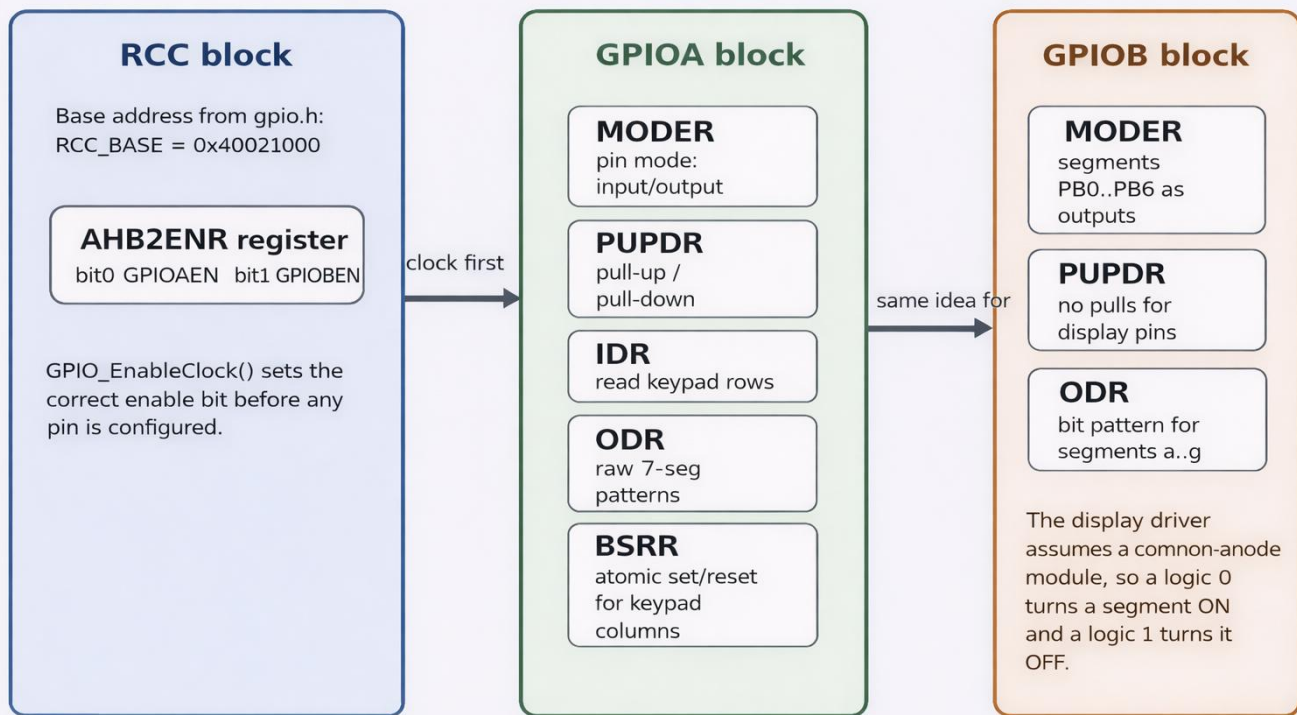


Figure 4. The driver reaches GPIO registers through typed pointers and fixed base addresses.

4.1 Base addresses defined in `gpio.h`

PERIPH_BASE	0x40000000	Start of peripheral space
RCC_BASE	0x40021000	RCC register block
GPIOA_BASE	0x48000000	GPIOA register block
GPIOB_BASE	0x48000400	GPIOB register block

4.2 GPIO registers that matter in this project

Register	Used in	Purpose	Example from project
RCC->AHB2ENR	GPIO_EnableClock()	Enable port clock	Set GPIOAEN or GPIOBEN

GPIOx->MODER	GPIO_InitPin()	Select input or output mode	Rows become input, columns become output
GPIOx->OTYPER	GPIO_InitPin()	Push-pull/open-drain type	Outputs stay push-pull
GPIOx->OSPEEDR	GPIO_InitPin()	Output speed	Cleared to low speed
GPIOx->PUPDR	GPIO_InitPin()	Internal pull-up or pull-down	Rows use pull-down
GPIOx->IDR	GPIO_ReadPin()	Read input state	Read keypad rows
GPIOx->BSRR	GPIO_WritePin()	Atomically set/reset bits	Drive one column high or low
GPIOx->ODR	SevenSeg_DisplayRaw()	Write full output pattern	Turn segments on or off

Why BSRR is excellent for GPIO writes: BSRR lets software set or reset a pin with one write, which is safer than reading ODR, modifying a bit, and writing it back. That is why the keypad columns are driven through GPIO_WritePin() and not by directly editing ODR.

4.3 How the structures map to hardware

- GPIO_TypeDef lists MODER, OTYPER, OSPEEDR, PUPDR, IDR, ODR, BSRR, and other registers in the same order as the real peripheral layout.
- RCC_TypeDef lists clock control registers such as AHB2ENR, which is the one needed to enable GPIOA and GPIOB.
- The macro GPIOA is a typed pointer to address 0x48000000, so GPIOA->MODER means 'go to the GPIOA peripheral and access its MODER register'.
- This style is common in embedded C because it keeps the source code readable while still producing direct register accesses.

5. Source tree overview

gpio.h	Header	Peripheral base addresses, register structures, enums, and function prototypes
gpio.c	Source	GPIO clock enabling, pin configuration, pin write, pin read
keypad.h	Header	Pin mapping and public keypad API
keypad.c	Source	Column scanning algorithm and key decoding
sevensseg.h	Header	Pin mapping and public display API
sevensseg.c	Source	Raw segment patterns and display helper functions
main.c	Source	Application logic, password checking, delay, and animation

6. File-by-file explanation

6.1 gpio.h

This header is the bridge between C code and the STM32L476RG hardware registers. It defines base addresses, register structures, useful GPIO enums, and the public function prototypes used by the rest of the project.

- __IO marks registers as volatile so the compiler always performs the memory access.
- __NOP() expands to a processor no-operation instruction and is used in the software delay routine.
- GPIO_TypeDef and RCC_TypeDef are structure layouts that mirror the real peripheral registers.

- GPIOA, GPIOB, and RCC are macros that convert base addresses into typed pointers.

API declared in gpio.h

- GPIO_EnableClock() - turns on the clock for the selected GPIO port.
- GPIO_InitPin() - configures one pin as input or output and selects pull resistors.
- GPIO_WritePin() - sets or resets one output pin through BSRR.
- GPIO_ReadPin() - reads the logic state of one input pin.

6.2 gpio.c

This source file implements the low-level GPIO driver used everywhere else in the project. Each function is small and focused.

GPIO_EnableClock()

```
void GPIO_EnableClock(GPIO_TypeDef *GPIOx);
```

Purpose: Enable the AHB2 clock for the requested GPIO port before any register in that port is accessed.

How it works: The function compares the pointer argument with GPIOA, GPIOB, GPIOC, and the other supported ports. When it finds a match, it sets the corresponding enable bit in RCC->AHB2ENR. The final read of RCC->AHB2ENR forces the write to complete before the function exits.

Parameters: GPIOx is the port pointer, for example GPIOA or GPIOB.

Registers involved: RCC->AHB2ENR.

GPIO_InitPin()

```
void GPIO_InitPin(GPIO_TypeDef *GPIOx, uint8_t pin, GPIO_Mode_t mode, GPIO_Pull_t pull);
```

Purpose: Configure one GPIO pin for input or output operation.

How it works: It first calls GPIO_EnableClock() so the port is active. Then it clears the two MODER bits for the selected pin and writes the requested mode. If the mode is output, it forces push-pull output type and low output speed. Finally, it programs the PUPDR bits to select no pull, pull-up, or pull-down.

Parameters: GPIOx selects the port, pin selects the bit number, mode selects input or output, and pull selects the internal resistor mode.

Registers involved: GPIOx->MODER, GPIOx->OTYPER, GPIOx->OSPEEDR, GPIOx->PUPDR.

GPIO_WritePin()

```
void GPIO_WritePin(GPIO_TypeDef *GPIOx, uint8_t pin, GPIO_PinState_t state);
```

Purpose: Drive one output pin high or low.

How it works: If the desired state is set, the function writes a 1 into the lower half of BSRR. If the desired state is reset, it writes a 1 into the upper half of BSRR at position pin+16. This makes the write atomic and avoids disturbing other pins in the same port.

Parameters: GPIOx is the destination port, pin is the output number, and state is GPIO_PIN_SET or GPIO_PIN_RESET.

Registers involved: GPIOx->BSRR.

GPIO_ReadPin()

```
GPIO_PinState_t GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint8_t pin);
```

Purpose: Read a single input pin and return a software-friendly state value.

How it works: The function checks whether the selected bit is high in IDR. If yes, it returns GPIO_PIN_SET; otherwise, it returns GPIO_PIN_RESET.

Parameters: GPIOx is the input port and pin is the bit number to test.

Return value: GPIO_PIN_SET if the bit is 1, otherwise GPIO_PIN_RESET.

Registers involved: GPIOx->IDR.

6.3 keypad.h

This header assigns physical pins to the keypad rows and columns and declares the public keypad driver interface.

- KEYPAD_PORT is GPIOA, so every keypad pin belongs to the same GPIO peripheral.
- The row pins are treated as inputs with pull-down resistors.
- The column pins are treated as outputs and are driven one at a time.
- KEYPAD_Init() prepares the GPIOs, and KEYPAD_GetKey() returns the pressed key as a character.

6.4 keypad.c

This file contains the keypad scanning algorithm. It is a blocking implementation: the function waits until a key is pressed and then waits again until the key is released. That behavior is simple and good for beginner projects because it avoids bounce-related double detection in most cases.

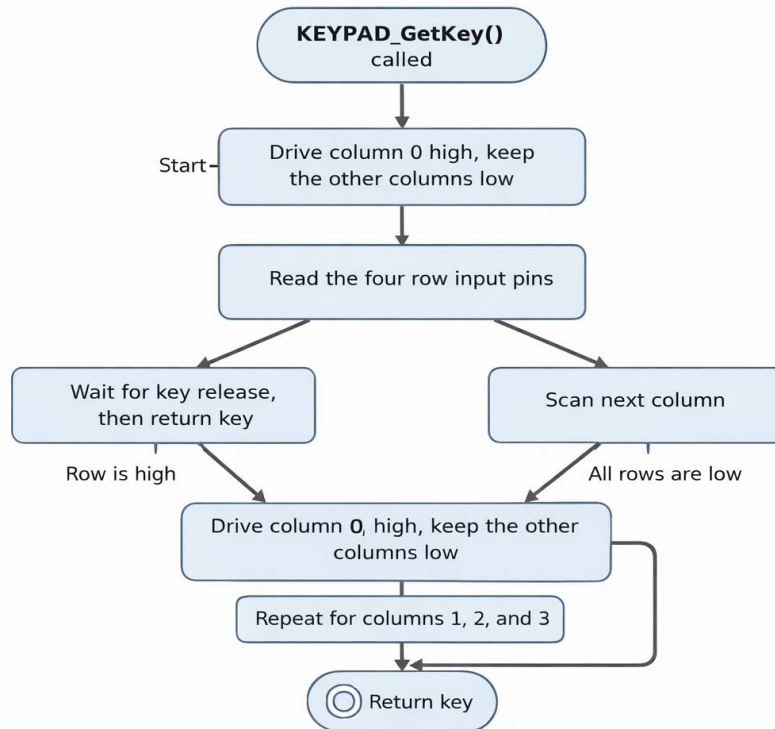


Figure 5. Scan flow used by KEYPAD_GetKey().

KEYPAD_AllColumnsLow()

```
static void KEYPAD_AllColumnsLow(void);
```

Purpose: Reset all four column outputs to logic 0.

How it works: It calls GPIO_WritePin() four times, once for each column pin. This creates a known default state before the driver energizes a single column.

Registers involved: GPIOA->BSRR through GPIO_WritePin().

KEYPAD_SetColumnHigh()

```
static void KEYPAD_SetColumnHigh(uint8_t col);
```

Purpose: Activate exactly one keypad column during the scan.

How it works: The function first calls KEYPAD_AllColumnsLow() to guarantee that only one column will be active. Then it sets the requested column pin high. If a key in that column is being pressed, the corresponding row input will read high.

Parameters: col is the column index 0 to 3.

Registers involved: GPIOA->BSRR through GPIO_WritePin().

KEYPAD_Init()

```
void KEYPAD_Init(void);
```

Purpose: Configure the keypad pins before the application starts scanning.

How it works: The four row pins are initialized as input pins with pull-down resistors so they stay low when no key is pressed. The four column pins are initialized as outputs with no pull resistors. The function finishes by driving all columns low.

Registers involved: GPIOA->MODER and GPIOA->PUPDR through GPIO_InitPin(); GPIOA->BSRR through KEYPAD_AllColumnsLow().

KEYPAD_GetKey()

```
char KEYPAD_GetKey(void);
```

Purpose: Scan the keypad continuously until a key press is detected, then return the matching character.

How it works: The function loops forever. For each column, it drives that column high, reads the four row inputs, and determines which key is pressed based on the active column and active row. When a match is found, it waits in a while loop until the key is released, clears all columns, and returns the character. The returned values include digits 0-9, letters A-D, and the special keys * and #.

Return value: The ASCII character corresponding to the pressed key.

Registers involved: GPIOA->BSRR and GPIOA->IDR through GPIO_WritePin() and GPIO_ReadPin().

6.5 sevenseg.h

This header maps display segments a to g onto GPIOB pins PB0 to PB6 and declares the public display functions.

- SevenSeg_Init() configures all display pins as outputs.
- SevenSeg_Clear() turns every segment off.
- SevenSeg_DisplayRaw() writes a raw 7-bit pattern directly to ODR.
- SevenSeg_DisplayDigit() displays numeric digits 0 to 9.
- SevenSeg_DisplayChar() handles digits plus A, B, C, D, * and #.

6.6 sevenseg.c

This source file contains the actual segment patterns and all display helper functions. It is a good example of how one low-level function can support several higher-level helper functions.

sevenSegDigits[]

```
static const uint8_t sevenSegDigits[10] = { ... };
```

Purpose: Store the raw common-anode bit patterns for digits 0 to 9.

How it works: Each array element uses bit 0 for segment a through bit 6 for segment g. Because the display is common-anode, a 0 bit turns a segment on.

Registers involved: Later copied into GPIOB->ODR by SevenSeg_DisplayRaw().

SevenSeg_Init()

```
void SevenSeg_Init(void);
```

Purpose: Configure PB0 to PB6 as outputs and clear the display.

How it works: The function calls GPIO_InitPin() once per segment pin, using output mode and no pull resistors. After all pins are configured, it calls SevenSeg_Clear() so the display starts in a known state.

Registers involved: GPIOB->MODER, GPIOB->OTYPER, GPIOB->OSPEEDR, GPIOB->PUPDR through GPIO_InitPin().

SevenSeg_Clear()

```
void SevenSeg_Clear(void);
```

Purpose: Turn all seven segments off.

How it works: It calls SevenSeg_DisplayRaw(0x7F). In this common-anode design, 0x7F means every segment bit is 1, so all segments are off.

Registers involved: GPIOB->ODR through SevenSeg_DisplayRaw().

SevenSeg_DisplayRaw()

```
void SevenSeg_DisplayRaw(uint8_t pattern);
```

Purpose: Write a raw 7-bit segment pattern to the display without disturbing the other GPIOB bits.

How it works: The function reads the current ODR value, clears the lowest seven bits with a mask, inserts the new pattern, and writes the result back. That preserves any GPIOB pins above PB6 if they were used for something else.

Parameters: pattern contains the desired states of segments a..g in bits 0..6.

Registers involved: GPIOB->ODR.

SevenSeg_DisplayDigit()

```
void SevenSeg_DisplayDigit(uint8_t digit);
```

Purpose: Display a decimal digit from 0 to 9.

How it works: If the argument is inside the valid range, the function looks up the matching pattern in sevenSegDigits[] and sends it to SevenSeg_DisplayRaw(). If the value is outside the range, the function clears the display instead of showing garbage.

Parameters: digit is a number from 0 to 9.

Registers involved: GPIOB->ODR through SevenSeg_DisplayRaw().

SevenSeg_DisplayChar()

```
void SevenSeg_DisplayChar(char c);
```

Purpose: Display one supported character on the 7-segment module.

How it works: A switch statement selects a pattern for '0' to '9', 'A', 'B', 'C', 'D', '*' and '#'. The star key is represented as a dash-like symbol and the hash key is represented with an underscore-like pattern. Any unsupported character clears the display.

Parameters: c is the character to be shown.

Registers involved: GPIOB->ODR through SevenSeg_DisplayRaw().

6.7 main.c

This file contains the application logic. It joins the keypad and display drivers into one behavior: read keys, store a four-character code, and validate it when the user presses *.

correctCode[]

```
static const char correctCode[CODE_LENGTH] = { 'A', '5', '2', 'D' };
```

Purpose: Store the only valid four-character password accepted by the program.

How it works: The array length is controlled by CODE_LENGTH, which keeps the code consistent if the password length changes later.

DelayMs()

```
static void DelayMs(volatile uint32_t ms);
```

Purpose: Provide a rough software delay in milliseconds.

How it works: The function uses a nested loop. The outer loop counts milliseconds, while the inner loop executes many NOP instructions. This is simple and good enough for display timing in a lab, but it is not a precise timing method because it depends on the system clock and compiler settings.

Parameters: ms is the approximate number of milliseconds to wait.

IsCodeCorrect()

```
static uint8_t IsCodeCorrect(const char *entered);
```

Purpose: Compare the entered code with the stored correct code.

How it works: A for loop checks each of the four positions. If any character differs, the function immediately returns 0. If all positions match, it returns 1.

Parameters: entered points to the user-entered character array.

Return value: 1 if the whole code matches, otherwise 0.

IsAllowedCodeKey()

```
static uint8_t IsAllowedCodeKey(char key);
```

Purpose: Decide whether a pressed key is allowed to become part of the password buffer.

How it works: Digits 0 to 9 and letters A to D are accepted. The special keys * and # are not stored as part of the password because they are reserved for submit and clear actions.

Parameters: key is the keypad character to test.

Return value: 1 for a valid password character, otherwise 0.

RunCircleAnimation()

```
static void RunCircleAnimation(void);
```

Purpose: Display a rotating outer-segment animation when the correct code is entered.

How it works: The function holds six raw segment patterns in circleFrames[]. A double loop sends each frame to the display and waits 50 ms, repeating the sequence four times. At the end, it clears the display.

Registers involved: GPIOB->ODR through SevenSeg_DisplayRaw().

main()

```
int main(void);
```

Purpose: Initialize the drivers and run the access-code application forever.

How it works: At startup the program initializes the keypad and 7-segment display, then clears the display. Inside the infinite loop it waits for a key, shows the key on the display, and interprets special actions. If the user presses #, the entered code is cleared. If the user presses *, the program checks whether exactly four allowed

characters were entered and whether they match the stored password. A correct code triggers `RunCircleAnimation()`; an incorrect code shows a dash briefly. Any valid alphanumeric key is added to the `enteredCode` buffer until four characters have been stored.

Return value: The function never returns during normal operation.

Very useful lab question: Ask students why the program displays the pressed key before it decides what to do with it. The answer is that immediate visual feedback makes the system easier to use and easier to debug.

7. End-to-end execution walkthrough

Power-up	<code>main()</code> calls <code>KEYPAD_Init()</code> and <code>SevenSeg_Init()</code> .	Display is blank.
Key press	<code>KEYPAD_GetKey()</code> scans columns and reads rows until a match is found.	Pressed key appears on the 7-segment display.
Normal character	If the key is 0-9 or A-D and there is room, it is stored in <code>enteredCode[]</code> .	Last pressed key remains visible.
Clear key #	<code>enteredCount</code> becomes 0 and the display is cleared.	Display turns off.
Enter key *	The code is checked only if exactly four characters were entered.	Either the animation runs or an error symbol appears.
Correct code	<code>RunCircleAnimation()</code> plays several raw segment frames.	Display looks like a rotating ring.
Wrong code	A dash is shown briefly, then the display is cleared.	User sees an error indication.

8. How to test the project in the lab

- Verify the wiring first, especially the keypad rows and columns.
- Build the project and flash it to the NUCLEO-L476RG.
- Press a single key and confirm that the matching symbol appears on the display.
- Press # and confirm that the display clears.
- Enter A, then 5, then 2, then D, and press *. The animation should run.
- Enter any wrong four-character sequence and press *. A short error indication should appear.

Common student mistakes: The most common problems are swapping keypad rows and columns, using a common-cathode display with common-anode patterns, forgetting current-limiting resistors, or not replacing the autogenerated `main.c` file.

9. Suggested student exercises

- Change the correct code and explain which lines must be updated.
- Add a visible success symbol before the circle animation starts.
- Replace the software delay with a timer-based delay.
- Add basic keypad debounce timing instead of relying only on the release wait loop.

- Extend the design to a two-digit multiplexed display.
- Log the entered keys through UART for debugging.

10. Conclusion

This project is a strong teaching example because it is small enough to understand in one lab session, yet rich enough to show real embedded-system ideas: clock enabling, GPIO configuration, input scanning, output encoding, state handling, and software structure. By creating a blank STM32CubeIDE project and then adding the files manually, students also learn how an embedded project is organized instead of treating it as a black box.